

“Autonomous Defense Grid Reconstruction System”

A futuristic research facility has deployed an **Autonomous Defense Grid (ADG)** to protect a classified laboratory.

The defense grid is organized hierarchically:

- Each **Defense Node** has:
 - A unique integer **NodeID**
 - A **ThreatScore**
 - A left sub-unit handling lower priority sectors
 - A right sub-unit handling higher priority sectors

The architecture obeys a strict **priority rule**:

- For every node:
 - All nodes in the left sub-unit have **NodeID** less than the parent.
 - All nodes in the right sub-unit have **NodeID** greater than the parent.

After a cyberattack, the system logs were partially damaged. You are provided with three different encrypted diagnostic reports generated by three different AI subsystems:

Report A – “Deployment Trace”

This report lists the order in which nodes were **activated during system initialization**.

The root control node is always activated first, followed recursively by its dependent sub-units.

Report B – “Stability Audit”

This report lists the nodes in the order in which the system verifies:

- all lower-priority sectors,
 - then the control node,
 - then higher-priority sectors.
-

Report C – “Decommission Log”

This report lists the nodes in the order in which they are **safely shut down**:

- Sub-units are shut down before their control node.
-

Your Task

You must implement a program in **C language** that:

Part 1: Reconstruction

1. Reconstruct the original Defense Grid structure using:
 - Report A and Report B
OR
 - Report B and Report C
 2. Validate that the reconstructed grid satisfies the strict priority rule.
-

Part 2: Analytical Operations

After reconstruction, your program must:

1. Generate all three reports again from the reconstructed structure.
 2. Determine:
 - Height of the grid
 - Total number of nodes
 - Whether the structure is perfectly balanced
 - The longest threat-propagation path
 3. Compute the sum of ThreatScores at each hierarchical level.
 4. Identify the critical node whose removal causes maximum structural imbalance.
-

Part 3: Advanced Constraint

You are **not allowed to use recursion** for generating any of the reports.

All depth-based operations must be implemented using:

- Explicit stack(s)
 - Or custom-designed iterative mechanisms
-

Input Format

N
NodeID ThreatScore (N lines)
ReportA (N integers)
ReportB (N integers)
ReportC (N integers)

Output Requirements

1. Confirmation of successful reconstruction or error message.
 2. All three regenerated reports.
 3. Height, balance status, longest path.
 4. Level-wise threat aggregation.
 5. Critical node ID.
-

Constraints

- $1 \leq N \leq 10^5$
- NodeID values are unique.
- Time Complexity must be $O(N)$
- Extra space complexity must be $O(N)$
- Recursion is strictly prohibited for traversal generation.